

You can directly use them for HID inputs, but you need to modify the code and prepare the code accordingly in that case.

In this project Arduino Uno has been used, so you need to do some tweaking to make it support HID. You need to install the flip software that you can get from the link <https://github.com/Pushpendra766/>

gaming-glove/tree/main/FLIP%20install

Next, you can convert the Arduino Uno into an HID device using the instructions and processes shown in a video available at <https://www.youtube.com/watch?v=tvqA-JcTQNg&feature=youtu.be>

Now you are ready to do the coding for the gaming glove. Arduino IDE

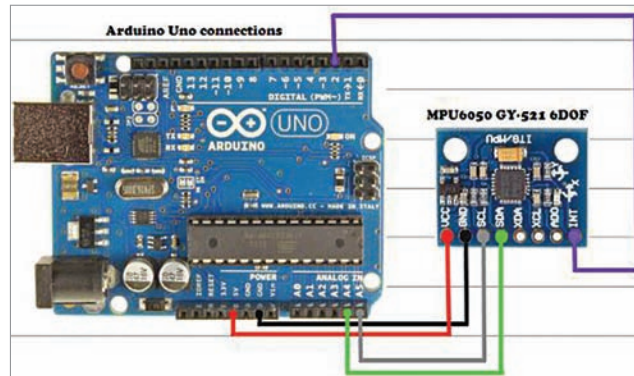


Fig. 3: Interconnection of the sensor and Arduino Uno

```

sketch_may08a $
#include<Wire.h>
uint8_t buf[8] = { 0 };

const int MPU_addr=0x68;
int16_t AcX, AcY, AcZ, Tmp, GyX, GyY, GyZ;

int minVal=265;
int maxVal=402;

double x;
double y;
double Xdiff;
double Ydiff;

bool runningX = true;
bool runningY = true;

void setup() {
  Wire.begin();
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission(true);
  Serial.begin(9600);
}

void loop() {
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_addr,14,true);
  AcX=Wire.read()<<8|Wire.read();
  AcY=Wire.read()<<8|Wire.read();
  AcZ=Wire.read()<<8|Wire.read();
  int xAng = map(AcX,minVal,maxVal,-90,90);
  int yAng = map(AcY,minVal,maxVal,-90,90);
  int zAng = map(AcZ,minVal,maxVal,-90,90);

```

Fig. 4: Code snippet getting MPU6050 values

```

void loop() {
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_addr,14,true);
  AcX=Wire.read()<<8|Wire.read();
  AcY=Wire.read()<<8|Wire.read();
  AcZ=Wire.read()<<8|Wire.read();
  int xAng = map(AcX,minVal,maxVal,-90,90);
  int yAng = map(AcY,minVal,maxVal,-90,90);
  int zAng = map(AcZ,minVal,maxVal,-90,90);

  x= RAD_TO_DEG * (atan2(-yAng, -zAng)+PI);
  y= RAD_TO_DEG * (atan2(-xAng, -zAng)+PI);

  Xdiff = (x<180)? x : (360-x);
  Ydiff = (y<160)? (y+20):abs(340 - y);

  if(x<5 || x>355){
    runningX = true;
  }
  if(y < 345 && y > 335){
    runningY = true;
  }

  if(Xdiff > Ydiff){
    //When button representing W is pressed
    if (x > 50 && x < 180 && runningX == true) {
      buf[2] = 4; // A keycode
      Serial.write(buf, 8); // Send keypress
      // Serial.println("A");
      releaseKey();
      runningX = false;
    }
  }
}

```

Fig. 5: Code snippet mapping the sensor value into HID mouse movement

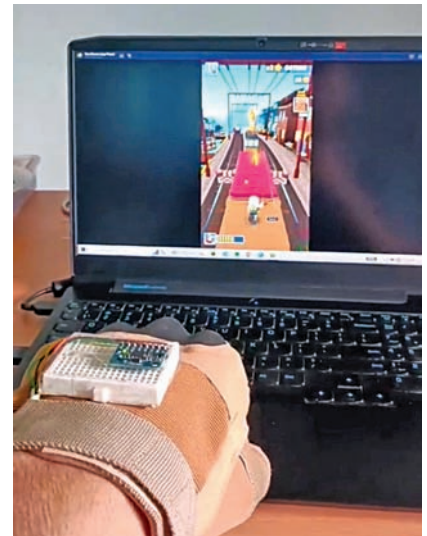


Fig. 6: Author testing the prototype

and z-axis movements into HID mouse movements, and also add the key functions, if needed. When the mouse movement or scroll is enabled, the calibrate function is

called to read the current tilt values. This function averages samples to get reliable offset values, which are used as the starting reference point. By subtracting these offsets from every ADC sample, the user can start at any position and use that hand orientation as the current reference axes.

After uploading the source code, you may assemble the project on a breadboard and fix it on top of a glove, as shown in Fig. 6. Now you can connect the Arduino Uno to USB and play any game virtually by just moving your hand in the air for an awesome experience. **EFY**

Pushpendra Chandravanshi, the author, is a passionate web developer and robotics enthusiast

